REVISION CHEAT SHEET:

Polymorphism in C#

POLYMORPHISM

Interview-ready notes · examples · C# · diagrams

Topic 14 of 40

Polymorphism is one of the core object-oriented programming ideas. The word means **many forms**: one common contract can represent many concrete implementations.

In C#, polymorphism appears most clearly in two places:

EXAMPLE / FLOW

```
Compile-time polymorphism
    ->
The compiler chooses which member to call

Runtime polymorphism
    ->
The runtime chooses which overridden/interface implementation to execute
```

In the Fundraising Platform, the same business operation may work with different runtime implementations:

EXAMPLE / FLOW

```
Payment processing
    -> CardPaymentProcessor
    -> AchPaymentProcessor
    -> MockPaymentProcessor

Receipt sending
    -> EmailReceiptSender
    -> PrintReceiptSender

Financial transaction
    -> GiftTransaction
    -> RefundTransaction
```

The caller can depend on a stable base type or interface while the actual runtime object supplies specialized behavior.

High-Level Mental Model

EXAMPLE / FLOW

```
One reference / one contract
        ↓
Many possible concrete objects
        ↓
Same call syntax
        ↓
Different behavior
```

Example:

```C#
FinancialTransaction transaction = new GiftTransaction();
string summary = transaction.GetSummary();
```

POLYMORPHISM

The variable is declared as `FinancialTransaction`, but the object is a `GiftTransaction`. If `GetSummary()` is virtual/abstract and overridden, the runtime executes the `GiftTransaction` implementation.

Topic Coverage

This topic covers:

- polymorphism definition;
- compile-time polymorphism;
- runtime polymorphism;
- method overloading;
- overload resolution;
- method overriding;
- `virtual`;
- `override`;
- abstract members and polymorphism;
- virtual dispatch;
- base references;
- derived objects;
- compile-time type vs runtime type;
- upcasting;
- interface polymorphism;
- base-class polymorphism;
- `new` member hiding vs overriding;
- `sealed override`;
- polymorphic properties, indexers, and events;
- object collections and polymorphism;
- polymorphism with dependency injection;
- substitutability;
- common mistakes;
- performance considerations;
- security considerations;
- ASP.NET Core usage;
- Fundraising Platform examples;
- interview questions;
- practical exercises.

What Is Polymorphism?

Definition

Polymorphism is the ability to use one common type or contract to work with objects that can provide different implementations of the same operation.

Instead of writing code that knows every concrete type:

C#

```
if (processorType == "Card")
{
    // Card logic.
}
else if (processorType == "ACH")
{
    // ACH logic.
}
```

POLYMORPHISM

polymorphic code can depend on a contract:

C#

```
await paymentProcessor.ProcessAsync(request, cancellationToken);
```

The concrete processor decides how the operation is performed.

Why Polymorphism Exists

Without polymorphism, high-level code often becomes full of type checks and branching:

EXAMPLE / FLOW

```
Caller
  -> if type A: do A logic
  -> if type B: do B logic
  -> if type C: do C logic
```

With polymorphism:

EXAMPLE / FLOW

```
Caller
  -> Common Contract
       -> A implementation
       -> B implementation
       -> C implementation
```

Benefits include:

EXAMPLE / FLOW

```
Lower coupling
Easier extension
Cleaner testing
Smaller condition trees
More reusable algorithms
Better dependency inversion
```

Two Main Forms in C#

Form	Decision Time	Typical Mechanism	Example
Compile-time polymorphism	Compile time	Method/operator overloading	`Calculate(int)` vs `Calculate(decimal)`
Runtime polymorphism	Runtime	`virtual` / `override`, abstract members, interfaces	`transaction.GetSummary()`

The most important distinction is **when the implementation is selected**.

Part 1 - Compile-Time Polymorphism

POLYMORPHISM

Compile-time polymorphism means the compiler can choose the target member before the program runs.

The main C# example is **method overloading**.

Method Overloading

Two or more methods may share the same name when their parameter lists are different.

C#

```
public sealed class GiftCalculator
{
    public decimal CalculateFee(decimal amount)
    {
        return amount * 0.02m;
    }

    public decimal CalculateFee(decimal amount, decimal rate)
    {
        return amount * rate;
    }
}
```

Calls:

C#

```
var calculator = new GiftCalculator();
decimal defaultFee = calculator.CalculateFee(1000m);
decimal customFee = calculator.CalculateFee(1000m, 0.015m);
```

The compiler determines which overload matches each call.

What Makes an Overload Different?

A method signature is distinguished primarily by:

EXAMPLE / FLOW

```
Method name
+
Number of parameters
+
Parameter types
+
Parameter order
+
Parameter passing modifiers where signature rules distinguish them
```

Examples:

C#

```
void Process(long giftId)
void Process(string externalReference)
void Process(long giftId, bool sendReceipt)
void Process(bool sendReceipt, long giftId)
```

These are distinct overloads.

Return Type Alone Cannot Overload

This is not valid:

```C#
// int GetValue()
// string GetValue()
```

The return type alone does not make a method signature unique for overload resolution.

Overload Resolution

At a call site, the compiler looks at the available candidates and determines the best applicable member.

```C#
static void Print(int value) { }
static void Print(decimal value) { }

Print(10);
Print(10m);
```

The first call selects `Print(int)`.

The second selects `Print(decimal)`.

Exact Match vs Conversion

Suppose:

```C#
static void Process(long value) { }
static void Process(decimal value) { }
```

Call:

```C#
Process(10);
```

The argument is an `int`, so the compiler evaluates which implicit conversion provides the best match.

Overload resolution can become difficult when many overloads rely on implicit conversions.

Avoid Ambiguous Overloads

Bad API design can create ambiguous calls:

```C#
void Send(string? address, object payload) { }
void Send(object target, string? message) { }
```

A call containing multiple `null` values can become ambiguous.

Prefer overload sets that are easy for humans and the compiler to distinguish.

Optional Parameters vs Overloads

You could write:

```C#
public void CreateGift(decimal amount, bool sendReceipt = true)
{
}
```

or:

```C#
public void CreateGift(decimal amount)
{
    CreateGift(amount, true);
}

public void CreateGift(decimal amount, bool sendReceipt)
{
}
```

Both can be useful.

Overloads are often clearer when the operations represent meaningfully different call shapes.

Optional parameters are often convenient when one value simply has a natural default.

Overloading Is Not Overriding

EXAMPLE / FLOW

```
Overloading
    -> Same method name
    -> Different parameter list
    -> Usually same type
    -> Compiler selects target

Overriding
    -> Same inherited virtual contract
    -> Compatible signature
    -> Derived type replaces behavior
    -> Runtime dispatch selects target
```

This is one of the most common interview comparisons.

Compile-Time Polymorphism Flow

EXAMPLE / FLOW

```
Source call
    ->
Compiler sees argument types
    ->
Compiler evaluates overload candidates
    ->
Best overload selected
    ->
Compiled call targets that member
```

Part 2 - Runtime Polymorphism

Runtime polymorphism occurs when the same base/interface call can execute different implementations depending on the actual runtime object.

Base Reference, Derived Object

```C#
FinancialTransaction transaction = new GiftTransaction();
```

Compile-time type:

```EXAMPLE / FLOW
FinancialTransaction
```

Runtime type:

```EXAMPLE / FLOW
GiftTransaction
```

This difference is central to runtime polymorphism.

Compile-Time Type vs Runtime Type

Question	Controlled By
Which members are directly visible to source code?	Compile-time type
Which overload is chosen?	Compile-time information
Which virtual override runs?	Runtime object type
Whether a cast can succeed	Runtime object type

Example:

```C#
FinancialTransaction transaction = new GiftTransaction();
```

The compiler lets you access members known to `FinancialTransaction`.

But a virtual call can execute a `GiftTransaction` override.

Upcasting

A derived object can be referenced by its base type.

```C#
GiftTransaction gift = new GiftTransaction();
FinancialTransaction transaction = gift;
```

This conversion is called **upcasting**.

It is normally implicit.

No second object is created.

```EXAMPLE / FLOW
GiftTransaction object
        ↑
GiftTransaction reference
        ↓ assign
FinancialTransaction reference
```

```
        ↑
Same object
```

POLYMORPHISM

Why Upcasting Matters

Upcasting lets one collection or method accept multiple specialized objects:

C#
```
List<FinancialTransaction> transactions =
[
    new GiftTransaction(),
    new RefundTransaction()
];
```

Each element can be processed through the same base contract.

Virtual Members

A base member participates in runtime overriding when it is declared `virtual`, `abstract`, or is already an overridable override.

Example:

C#
```
public abstract class FinancialTransaction
{
    public virtual string GetSummary()
    {
        return "Financial transaction";
    }
}
```

Derived:

C#
```
public sealed class GiftTransaction : FinancialTransaction
{
    public override string GetSummary()
    {
        return "Gift transaction";
    }
}
```

Runtime Dispatch

C#
```
FinancialTransaction transaction = new GiftTransaction();
string summary = transaction.GetSummary();
```

The runtime executes:

EXAMPLE / FLOW
```
GiftTransaction.GetSummary()
```

because the object is a `GiftTransaction`.

Virtual Dispatch Flow

EXAMPLE / FLOW

```
Base reference
    -> FinancialTransaction

Runtime object
    -> GiftTransaction

Call virtual member
    -> GetSummary()

Runtime checks most-derived override
    -> GiftTransaction.GetSummary()
```

Override Rules

For normal class overriding:

- the base member must be overridable (`virtual`, `abstract`, or an overridable `override`);
- the derived member uses `override`;
- the contract must remain signature-compatible;
- a `sealed override` cannot be overridden again;
- fields do not participate in virtual overriding;
- methods, properties, indexers, and events can participate in virtual behavior.

Abstract Members

An abstract member declares required behavior without providing a base implementation.

C#

```
public abstract class FinancialTransaction
{
    public abstract string GetSummary();
}
```

Derived types must implement it unless they remain abstract:

C#

```
public sealed class RefundTransaction : FinancialTransaction
{
    public override string GetSummary()
    {
        return "Refund transaction";
    }
}
```

Abstract members are therefore naturally polymorphic.

Virtual vs Abstract

virtual	abstract
Base implementation exists	No ordinary base implementation
Override is optional	Concrete derived implementation required

virtual	abstract
Base behavior can be reused	Derived type must supply behavior
Good for optional specialization	Good for required specialization

Calling Base Behavior from an Override

```C#
public override string GetSummary()
{
    string baseSummary = base.GetSummary();
    return $"{baseSummary} - Gift";
}
```

Use `base.Member()` when the derived behavior intentionally includes the base implementation.

Do not call it automatically if the semantics do not require it.

Polymorphic Properties

```C#
public abstract class FinancialTransaction
{
    public virtual string DisplayName => "Transaction";
}

public sealed class GiftTransaction : FinancialTransaction
{
    public override string DisplayName => "Gift";
}
```

Call:

```C#
FinancialTransaction transaction = new GiftTransaction();
Console.WriteLine(transaction.DisplayName);
```

Result:

EXAMPLE / FLOW

```
Gift
```

Indexers and Events

C# also allows indexers and events to participate in virtual/override behavior.

The mental model remains:

EXAMPLE / FLOW

```
Base virtual contract
    ->
Derived override
    ->
Runtime dispatch
```

Member Hiding with new

`new` is not runtime overriding.

```C#
public class BaseReport
{
    public void Print()
    {
        Console.WriteLine("Base");
    }
}

public class GiftReport : BaseReport
{
    public new void Print()
    {
        Console.WriteLine("Gift");
    }
}
```

Calls:

```C#
GiftReport giftReport = new();
giftReport.Print();

BaseReport baseReport = giftReport;
baseReport.Print();
```

The selected member depends on the compile-time reference type because the method is hidden, not overridden.

override vs new

override	new
Continues virtual dispatch	Hides inherited member
Runtime object type matters	Compile-time reference type matters
Represents polymorphic specialization	Represents a separate member with same name
Requires overridable base member	Can hide nonvirtual member

Call Matrix - Override

```C#
public class Base
{
    public virtual string Name() => "Base";
}

public class Derived : Base
{
    public override string Name() => "Derived";
}
```

Variable	Object	Result
`Base`	`Base`	`Base`
`Derived`	`Derived`	`Derived`
`Base`	`Derived`	`Derived`

POLYMORPHISM

Runtime dispatch follows the object.

Call Matrix - Hiding

```C#
public class Base
{
    public string Name() => "Base";
}

public class Derived : Base
{
    public new string Name() => "Derived";
}
```

Variable	Object	Result
`Base`	`Base`	`Base`
`Derived`	`Derived`	`Derived`
`Base`	`Derived`	`Base`

Member hiding follows the compile-time type.

sealed override

A derived class can stop further overriding:

```C#
public class GiftTransaction : FinancialTransaction
{
    public sealed override string GetSummary()
    {
        return "Gift";
    }
}
```

Further derived classes cannot override that virtual slot again.

Use this when the behavior must become final at a particular level.

Interface Polymorphism

Interfaces provide another major runtime-polymorphism mechanism.

```C#
public interface IReceiptSender
{
    Task SendAsync(Gift gift, CancellationToken cancellationToken);
}
```

Implementations:

POLYMORPHISM

C#
```
public sealed class EmailReceiptSender : IReceiptSender
{
    public Task SendAsync(Gift gift, CancellationToken cancellationToken)
    {
        Console.WriteLine("Email receipt");
        return Task.CompletedTask;
    }
}
```

C#
```
public sealed class PrintReceiptSender : IReceiptSender
{
    public Task SendAsync(Gift gift, CancellationToken cancellationToken)
    {
        Console.WriteLine("Print receipt");
        return Task.CompletedTask;
    }
}
```

Caller:

C#
```
IReceiptSender sender = new EmailReceiptSender();
await sender.SendAsync(gift, cancellationToken);
```

Later:

C#
```
IReceiptSender sender = new PrintReceiptSender();
await sender.SendAsync(gift, cancellationToken);
```

Same contract.

Different runtime implementation.

Interface Polymorphism Flow

EXAMPLE / FLOW
```
Caller
    -> IReceiptSender
        -> EmailReceiptSender
        -> PrintReceiptSender
        -> TestReceiptSender
```

The caller does not need a type-checking chain.

Base-Class vs Interface Polymorphism

Base-Class Polymorphism	Interface Polymorphism
Uses inheritance hierarchy	Uses implemented contract
Can share state/implementation	Usually focuses on capability
One direct base class	Multiple interfaces possible

Base-Class Polymorphism	Interface Polymorphism
`virtual` / `override` common	Interface dispatch common
Good for true IS-A relationships	Good for replaceable capabilities

POLYMORPHISM

Interface Implementation and Virtual Overrides

A class can implement an interface with a virtual method, and a derived class can override that method.

C#

```
public interface IProcessor
{
    void Process();
}

public class BaseProcessor : IProcessor
{
    public virtual void Process()
    {
        Console.WriteLine("Base");
    }
}

public sealed class GiftProcessor : BaseProcessor
{
    public override void Process()
    {
        Console.WriteLine("Gift");
    }
}
```

Through an `IProcessor` reference to a `GiftProcessor`, the override can execute.

Explicit Interface Implementation

An explicit interface implementation is callable through the interface reference.

C#

```
public interface IAuditable
{
    string GetAuditDescription();
}

public sealed class Gift : IAuditable
{
    string IAuditable.GetAuditDescription()
    {
        return "Gift audit";
    }
}
```

Call:

C#

```
IAuditable auditable = new Gift();
string description = auditable.GetAuditDescription();
```

Explicit interface members are a different mechanism from class `virtual` / `override` members.

Polymorphic Collections

A major benefit is processing heterogeneous objects uniformly.

```C#
List<FinancialTransaction> transactions =
[
    new GiftTransaction(),
    new RefundTransaction()
];

foreach (FinancialTransaction transaction in transactions)
{
    Console.WriteLine(transaction.GetSummary());
}
```

Each object chooses its override at runtime.

Why Polymorphic Collections Matter

Without polymorphism:

```EXAMPLE / FLOW
List<object>
    -> type checks
    -> casts
    -> switch on runtime type
```

With a good common contract:

```EXAMPLE / FLOW
List<FinancialTransaction>
    -> transaction.GetSummary()
```

The algorithm stays stable as more transaction types are added.

Fundraising Example - Payment Processing

Contract:

```C#
public interface IPaymentProcessor
{
    Task<PaymentResult> ProcessAsync(
        PaymentRequest request,
        CancellationToken cancellationToken);
}
```

Card implementation:

```C#
public sealed class CardPaymentProcessor : IPaymentProcessor
{
    public Task<PaymentResult> ProcessAsync(
        PaymentRequest request,
        CancellationToken cancellationToken)
    {
        return Task.FromResult(PaymentResult.Success());
```

```
    }
}
```

POLYMORPHISM

ACH implementation:

```C#
public sealed class AchPaymentProcessor : IPaymentProcessor
{
    public Task<PaymentResult> ProcessAsync(
        PaymentRequest request,
        CancellationToken cancellationToken)
    {
        return Task.FromResult(PaymentResult.Success());
    }
}
```

Service:

```C#
public sealed class GiftPaymentService
{
    private readonly IPaymentProcessor _paymentProcessor;

    public GiftPaymentService(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }

    public Task<PaymentResult> ProcessAsync(
        PaymentRequest request,
        CancellationToken cancellationToken)
    {
        return _paymentProcessor.ProcessAsync(request, cancellationToken);
    }
}
```

The service is polymorphic because a different implementation can be injected without changing the service algorithm.

Polymorphism with Dependency Injection

EXAMPLE / FLOW

```
Controller
    -> IGiftService
        -> GiftService
            -> IPaymentProcessor
                -> CardPaymentProcessor
                -> AchPaymentProcessor
```

Dependency injection does not create polymorphism by itself.

It makes polymorphic substitution convenient by supplying different implementations of a common contract.

ASP.NET Core Example

```C#
builder.Services.AddScoped<IPaymentProcessor, CardPaymentProcessor>();
```

Consumer:


```C#
public sealed class GiftsController : ControllerBase
{
    private readonly IPaymentProcessor _paymentProcessor;

    public GiftsController(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }
}
```

The controller depends on the interface, not the concrete processor.

Substitutability

Polymorphism is useful only when implementations honor the same meaningful contract.

If an interface says:

```C#
Task SaveAsync(Gift gift, CancellationToken cancellationToken);
```

one implementation should not secretly delete the gift.

Matching method signatures is not enough.

Implementations should preserve the behavioral meaning expected by callers.

Liskov Substitution Preview

A useful design test is:

EXAMPLE / FLOW

```
Can every implementation be used
where the abstraction is expected
without surprising the caller?
```

If not, the abstraction may be poorly designed.

This idea will be explored more deeply in the SOLID topic.

Polymorphism vs Type Checking

Sometimes code grows like this:

```C#
if (transaction is GiftTransaction gift)
{
    // Gift behavior.
}
else if (transaction is RefundTransaction refund)
{
    // Refund behavior.
}
```

If the operation naturally belongs to the transaction type, a virtual or interface method may be cleaner:

C#

```
transaction.Process();
```

POLYMORPHISM

However, pattern matching is appropriate when behavior belongs to the caller rather than the object, or when you intentionally need type-specific data.

Polymorphism vs Pattern Matching

Do not assume one always replaces the other.

Use polymorphism when:

EXAMPLE / FLOW

```
Behavior belongs to the abstraction
Implementations own their specialized behavior
You want extensibility through new implementations
```

Use pattern matching when:

EXAMPLE / FLOW

```
The caller owns the decision
A closed set of cases is acceptable
You need shape/type-specific decomposition
```

Object Methods and Polymorphism

Normal classes inherit virtual members such as `ToString()`, `Equals(object?)`, and `GetHashCode()` from `object`.

Types may override them.

Example:

C#

```
public sealed class GiftReference
{
    public required long GiftId { get; init; }

    public override string ToString()
    {
        return $"GIFT-{GiftId}";
    }
}
```

A call through an `object` reference can still execute the override.

Constructors Are Not Polymorphic

Constructors are not virtual and are not overridden.

EXAMPLE / FLOW

```
new GiftTransaction()
```

selects a constructor based on the constructed type and arguments.

Inheritance constructor chaining is a different concept from runtime method dispatch.

Fields Are Not Polymorphic

Fields cannot be `virtual` or `override`.

If polymorphic state access is required, expose behavior through a virtual property or method.

Static Members Are Not Runtime Polymorphic

Static members belong to the type, not an object instance.

C#
```
public class Base
{
    public static void Print() { }
}

public class Derived : Base
{
    public new static void Print() { }
}
```

Selection depends on the type used at the call site, not on runtime object dispatch.

Common Mistake - Confusing Overloading and Overriding

Wrong mental model:

EXAMPLE / FLOW
```
Same method name = overriding
```

Correct:

EXAMPLE / FLOW
```
Overloading
    -> different parameter lists

Overriding
    -> inherited virtual contract
```

Common Mistake - Forgetting virtual

This cannot be overridden:

C#
```
public void Process()
{
}
```

A derived class cannot write:

C#
```
// public override void Process()
```

unless the inherited member is overridable.

Common Mistake - Using new When override Is Intended

```C#
public new void Process()
{
}
```

creates hiding behavior.

A base reference can still call the base member.

If runtime polymorphism is intended, use the proper virtual/override contract.

Common Mistake - Assuming Base Reference Loses Derived Object

```C#
FinancialTransaction transaction = new GiftTransaction();
```

does not convert the object into a base object.

The object is still a `GiftTransaction`.

Only the reference's compile-time view is `FinancialTransaction`.

Common Mistake - Downcasting to Reach Every Behavior

If code constantly does:

```C#
GiftTransaction gift = (GiftTransaction)transaction;
```

ask whether the missing behavior should be part of the abstraction.

Repeated casting can be a sign that the polymorphic contract is incomplete.

Common Mistake - Blind Downcast

```C#
FinancialTransaction transaction = new RefundTransaction();
GiftTransaction gift = (GiftTransaction)transaction;
```

This can throw `InvalidCastException`.

Prefer pattern matching when a runtime type check is genuinely required:

```C#
if (transaction is GiftTransaction gift)
{
    // Safe use.
}
```

Common Mistake - Giant Base Class

Polymorphism does not justify placing every possible operation in a base class.

Bad:

EXAMPLE / FLOW

```
FinancialTransaction
    -> EmailReceipt()
    -> ExportPdf()
    -> ChargeCard()
    -> PrintLabel()
    -> ReconcileBank()
```

Those are unrelated responsibilities.

Prefer focused abstractions and composition.

Common Mistake - Interface per Class Without Meaning

Creating:

EXAMPLE / FLOW

```
IGiftFormatter
GiftFormatter
```

for every tiny class does not automatically improve polymorphism.

Use an interface when meaningful substitution or a boundary exists.

Common Mistake - Violating Contract Meaning

Suppose:

C#

```
IPaymentProcessor.ProcessAsync(...)
```

is expected to return a result without throwing for normal card declines.

If one implementation throws for every decline while another returns a failed result, callers may not be safely substitutable.

Polymorphism requires semantic consistency.

Performance Considerations

Virtual/interface calls can add a dispatch step compared with a statically known nonvirtual call.

For ordinary ASP.NET Core business code, the cost is usually tiny compared with:

EXAMPLE / FLOW

```
Database I/O
HTTP calls
Network latency
Serialization
Disk access
```

The JIT may devirtualize some calls when the runtime type is known.

Do not remove useful abstractions based on assumed micro-performance differences. Measure first.

Allocation Considerations

Polymorphism with classes normally passes object references; assigning a derived object to a base reference does not create a second object.

```C#
FinancialTransaction transaction = giftTransaction;
```

POLYMORPHISM

copies a reference.

For value types converted to interface/object references, boxing can occur in relevant scenarios. That concern belongs mainly to value-type and generics performance topics.

Security Considerations

A polymorphic abstraction can reduce exposure by giving callers only the capability they need.

Example:

```C#
public interface ISecretValidator
{
    bool Validate(string suppliedValue);
}
```

The caller does not need the actual secret.

However, the fact that a type implements a trusted-looking interface does not make the implementation trustworthy.

Security-sensitive implementations still need controlled registration, configuration, review, and authorization.

Security - Do Not Trust User-Selected Implementation Names

Avoid accepting arbitrary external type names such as:

EXAMPLE / FLOW

```
processorType = "AdministratorBypassProcessor"
```

and dynamically instantiating them without an allow-list and controlled configuration.

Polymorphic extensibility should not become an untrusted code-selection mechanism.

Testing Polymorphic Code

A major benefit is substitution in tests.

Production:

```C#
IPaymentProcessor processor = new CardPaymentProcessor(...);
```

Test:

```C#
IPaymentProcessor processor = new FakePaymentProcessor();
```

The service can be tested without real external payment infrastructure.

Fundraising Example - Transaction Hierarchy

```C#
public abstract class FinancialTransaction
{
    protected FinancialTransaction(long transactionId, decimal amount)
    {
        TransactionId = transactionId;
        Amount = amount;
    }

    public long TransactionId { get; }
    public decimal Amount { get; }

    public abstract string GetSummary();
}
```

Gift:

```C#
public sealed class GiftTransaction : FinancialTransaction
{
    public GiftTransaction(long transactionId, decimal amount, long constituentId)
        : base(transactionId, amount)
    {
        ConstituentId = constituentId;
    }

    public long ConstituentId { get; }

    public override string GetSummary()
    {
        return $"Gift {TransactionId} for constituent {ConstituentId}: {Amount:C2}";
    }
}
```

Refund:

```C#
public sealed class RefundTransaction : FinancialTransaction
{
    public RefundTransaction(long transactionId, decimal amount, long originalGiftId)
        : base(transactionId, amount)
    {
        OriginalGiftId = originalGiftId;
    }

    public long OriginalGiftId { get; }

    public override string GetSummary()
    {
        return $"Refund {TransactionId} for gift {OriginalGiftId}: {Amount:C2}";
    }
}
```

Processing the Hierarchy

```C#
FinancialTransaction[] transactions =
[
```

```
    new GiftTransaction(1, 1000m, 5001),
    new RefundTransaction(2, 200m, 9001)
];

foreach (FinancialTransaction transaction in transactions)
{
    Console.WriteLine(transaction.GetSummary());
}
```

POLYMORPHISM

The loop does not need a type switch.

Complete Interface Example

C#

```
public interface IGiftReceiptSender
{
    Task SendAsync(Gift gift, CancellationToken cancellationToken);
}

public sealed class EmailGiftReceiptSender : IGiftReceiptSender
{
    public Task SendAsync(Gift gift, CancellationToken cancellationToken)
    {
        Console.WriteLine($"Email receipt for gift {gift.GiftId}");
        return Task.CompletedTask;
    }
}

public sealed class NoOpGiftReceiptSender : IGiftReceiptSender
{
    public Task SendAsync(Gift gift, CancellationToken cancellationToken)
    {
        return Task.CompletedTask;
    }
}
```

Service:

C#

```
public sealed class GiftService
{
    private readonly IGiftReceiptSender _receiptSender;

    public GiftService(IGiftReceiptSender receiptSender)
    {
        _receiptSender = receiptSender;
    }

    public Task SendReceiptAsync(Gift gift, CancellationToken cancellationToken)
    {
        return _receiptSender.SendAsync(gift, cancellationToken);
    }
}
```

Design Decision Flow

EXAMPLE / FLOW

```
Do multiple implementations share one meaningful contract?
    ↓
Yes
    ↓
```

```
Should callers be independent of concrete type?
    ↓
Yes
    ↓
Use polymorphic abstraction
```

POLYMORPHISM

Then ask:

EXAMPLE / FLOW

```
Do types share true IS-A identity/state/implementation?
    -> Base class may fit

Do types mainly share a capability?
    -> Interface may fit
```

Practical Exercise 1 - Overloading

Create three overloads:

C#

```
CalculateFee(decimal amount)
CalculateFee(decimal amount, decimal rate)
CalculateFee(decimal amount, decimal rate, decimal minimumFee)
```

Then explain which overload is chosen for each call.

Practical Exercise 2 - Runtime Dispatch

Create a base `Notification` class with virtual `Send()` and derived `EmailNotification` and `SmsNotification` classes.

Store both in:

C#

```
List<Notification>
```

Call `Send()` in a loop and observe the runtime-selected override.

Practical Exercise 3 - override vs new

Create two versions of the same example:

1. base `virtual` + derived `override`;
2. base nonvirtual + derived `new`.

Call through both base and derived references and compare results.

Practical Exercise 4 - Interface Polymorphism

Define:

C#

```
public interface IExportWriter
{
    string Write(Gift gift);
}
```

Implement:

EXAMPLE / FLOW

```
CsvGiftWriter
JsonGiftWriter
```

POLYMORPHISM

Use a single `IExportWriter` variable to call both implementations.

Practical Exercise 5 - Fundraising Transactions

Add a third derived transaction:

EXAMPLE / FLOW

```
AdjustmentTransaction
```

without changing the loop that processes `FinancialTransaction` objects.

Practical Exercise 6 - Dependency Injection

Inject:

EXAMPLE / FLOW

```
IGiftReceiptSender
```

into a service.

Use a production implementation and a fake test implementation.

Interview Questions

1. What is polymorphism?

Polymorphism is the ability to use a common base type or interface to work with different concrete implementations while allowing the appropriate behavior to be selected.

2. What are the main forms of polymorphism in C#?

A common practical distinction is compile-time polymorphism through overloading and runtime polymorphism through virtual overriding or interface dispatch.

3. What is compile-time polymorphism?

The compiler selects the target member before execution based on the available overloads and compile-time argument information.

4. What is runtime polymorphism?

A call through a base/interface reference executes behavior selected from the runtime object's implementation.

5. What is method overloading?

Multiple methods share a name but have different parameter lists.

6. Can methods be overloaded only by return type?

No. Return type alone does not distinguish overloads.

7. What is method overriding?

A derived class provides a specialized implementation of an inherited overridable member using `override`.

8. What must a base method have to be overridden?

It must be `virtual`, `abstract`, or an overridable `override`, and it must not be sealed against further overriding.

9. What is virtual dispatch?

The runtime selects the most appropriate override based on the actual runtime type of the object.

10. What is the difference between compile-time type and runtime type?

The compile-time type controls what members the compiler exposes and participates in overload resolution. The runtime type controls virtual dispatch and runtime casting behavior.

11. What is upcasting?

Assigning a derived reference to a base/interface reference. It is normally implicit and does not create a new object.

12. What is downcasting?

Converting a base reference back to a more derived type. It can fail if the runtime object is not of that type.

13. What is safer than a blind downcast?

Pattern matching such as `if (value is GiftTransaction gift)`.

14. What is the difference between override and new?

`override` participates in runtime virtual dispatch. `new` hides an inherited member and selection depends on the compile-time reference type.

15. Can fields be virtual?

No. Fields do not participate in virtual overriding.

16. Can properties be virtual and overridden?

Yes. Properties, methods, indexers, and events can participate in virtual behavior.

17. Are constructors polymorphic?

No. Constructors are not virtual or overridden.

18. Are static methods runtime polymorphic?

No. Static member selection is based on the type used at the call site rather than runtime object dispatch.

19. What is interface polymorphism?

A caller uses an interface reference while different implementing runtime types supply the behavior.

20. Why is interface polymorphism useful with dependency injection?

Services can depend on contracts while the DI container supplies different implementations for production, testing, or configuration.

21. What is substitutability?

An implementation should be safely usable wherever its abstraction is expected without violating the caller's assumptions.

22. Does polymorphism eliminate every type check?

No. Pattern matching and type checks still have legitimate uses when the caller owns type-specific behavior.

23. What is a sealed override?

An override that continues to satisfy the virtual contract but prevents further derived classes from overriding that member.

24. Does a base reference change a derived object into a base object?

No. The object retains its runtime type; only the reference's compile-time view changes.

25. What happens when a virtual method is called through a base reference that points to a derived object?

The runtime selects the most-derived applicable override.

26. What happens when a hidden nonvirtual method is called through a base reference?

The base member is selected because hiding follows the compile-time reference type.

27. Why can too many virtual members be risky?

Every virtual member becomes an extension point and part of the subclass contract, increasing maintenance and compatibility obligations.

28. Is polymorphism mainly about code reuse?

No. Its more important purpose is substitutability and stable contracts across varying implementations.

29. Does an interface automatically make code loosely coupled?

Not necessarily. The contract must be meaningful and consumers should depend only on what they need.

30. What is the main difference between overloading and overriding?

Overloading is compile-time member selection among different signatures. Overriding is runtime specialization of an inherited virtual contract.

Best Practices

- Use polymorphism when multiple implementations truly share one meaningful contract.
- Prefer interfaces for replaceable capabilities across unrelated types.
- Prefer a base class when there is a genuine IS-A hierarchy with shared state/implementation.
- Mark members `virtual` only when specialization is intentional.

- Use `override` for true runtime specialization.
- Use `new` only when hiding is deliberate and understood.
- Keep polymorphic contracts small and cohesive.
- Avoid repeated downcasting when a missing operation should belong to the abstraction.
- Preserve behavioral meaning across implementations.
- Use dependency injection to supply implementations, not to justify unnecessary interfaces.
- Keep security-sensitive implementation selection controlled.
- Measure performance before removing useful abstractions.
- Prefer readable contracts over giant type switches when behavior naturally belongs to the implementation.

Quick Revision

Concept	Meaning
Polymorphism	One contract, many implementations
Compile-time polymorphism	Compiler selects overload
Runtime polymorphism	Runtime selects override/interface implementation
Overloading	Same name, different parameter list
Overriding	Derived implementation of virtual/abstract member
`virtual`	Base member may be overridden
`override`	Derived virtual specialization
`new`	Hides inherited member
Base reference	Variable typed as base
Runtime object	Actual concrete object
Upcast	Derived -> base/interface
Downcast	Base -> derived
Virtual dispatch	Runtime chooses override
Interface polymorphism	Interface reference -> concrete implementation
`sealed override`	Stops further overriding
Substitutability	Implementation safely behaves as abstraction

Final End-to-End Concept Flow

EXAMPLE / FLOW

```
Compile-time polymorphism
    -> Overloaded members
    -> Compiler examines arguments
    -> Best overload selected
```

EXAMPLE / FLOW

```
Runtime polymorphism
    -> Base/interface reference
    -> Derived/concrete runtime object
    -> Virtual/interface call
```

```
    -> Runtime dispatch
    -> Concrete implementation executes
```

POLYMORPHISM

Fundraising example:

EXAMPLE / FLOW

```
GiftService
    -> IPaymentProcessor
        -> CardPaymentProcessor
        -> AchPaymentProcessor
        -> FakePaymentProcessor
```

and:

EXAMPLE / FLOW

```
FinancialTransaction
    -> GiftTransaction
    -> RefundTransaction
```

The core lesson is:

Polymorphism lets callers program against stable contracts while concrete implementations vary. Overloading is chosen by the compiler, while overriding and interface dispatch select behavior at runtime. Use polymorphism when implementations are genuinely substitutable, not merely because several classes happen to have similar method names.

Next roadmap topic: Topic 15 - Abstraction and Interfaces.

References

- Microsoft Learn - Polymorphism in C#.
- Microsoft Learn - Inheritance in C#.
- Microsoft Learn - Common Type System and method overloading.
- Microsoft C# language specification - classes and interfaces.